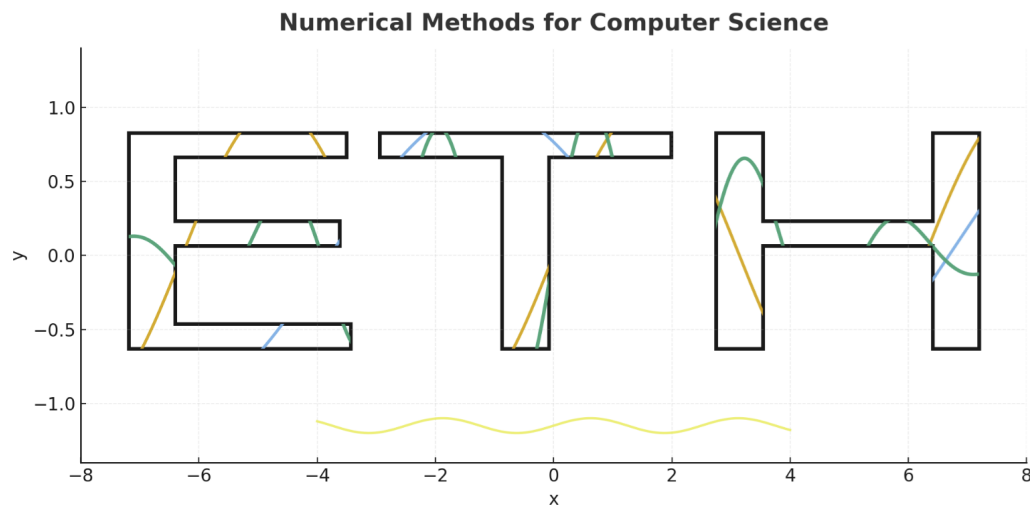




Applied Linear Algebra — Week #10

ETH Zurich

DAVID Mihnea-Ștefan

mihdavid@ethz.ch

This week introduces key ideas from applied linear algebra, focusing on solving systems, understanding projections, and exploring how matrix methods underpin practical computation and data analysis.

Contents

1	Motivation and Overview	1
1.1	Why We Need It	1
1.2	From Radar Measurements to Linear Systems	3
2	Linear Algebra Recap	5
2.1	Vectors, Matrices, and Columns	5
2.2	The Four Fundamental Subspaces	6
2.3	Types of Linear Systems	7
3	Square LSE	10
3.1	Problem Setup	10
3.2	Augmented Form and Row Operations	10
3.3	Gaussian Elimination (GE)	13
3.4	Permutation and Factorization: from Row Operations to LU Decomposition	14
3.5	Block Gaussian Elimination	17
3.6	Computing the Matrix Inverse in Practice	19
3.7	Python Snippets (Direct Solve)	20
4	Exploiting Structure in Solving Linear Systems	21
4.1	Example 1: Arrow Matrices	21
4.2	Example 2: Low-Rank Modifications and the Sherman–Morrison–Woodbury Formula	22

TA Award 2025

A note from your TA



Hello everyone! As you've probably already seen, VIS is once again organizing the **TA Award**, a prize given by students to the TA who resonated the most with them. If you feel that this might be me, and that I've managed, even a little, to make the NumCS course more digestible or interesting, I would be truly grateful if you considered voting for me.

On your side, it only takes a minute to open the link and cast your vote, but for me it would be a real confirmation that what I do actually has an impact. Knowing that my work has helped you, even in a small way, would mean a lot.

Thank you very much for your time and support!

1 Motivation and Overview

Linear algebra shows up almost everywhere in computer science — often in places where you wouldn't expect it. Whenever you deal with data, graphics, or simulations, you are secretly using it.

Think about a few examples:

- Turning or zooming a 3D model in a game — that's a matrix changing all points at once.
- Training a machine learning model — that's solving for numbers that best fit your data.
- Compressing an image — that's breaking a big matrix into smaller, more meaningful parts.
- Ranking websites or users — that's computing eigenvectors in a huge graph.

All these different problems share the same mathematical heart. Linear algebra gives us a way to describe many related values at once, and to understand how changing one affects the others.

Note

Linear algebra lets us think in *bulk*. Instead of updating one number at a time, we describe entire systems with a single equation. What would take hundreds of lines of code in a loop becomes one clean matrix operation.

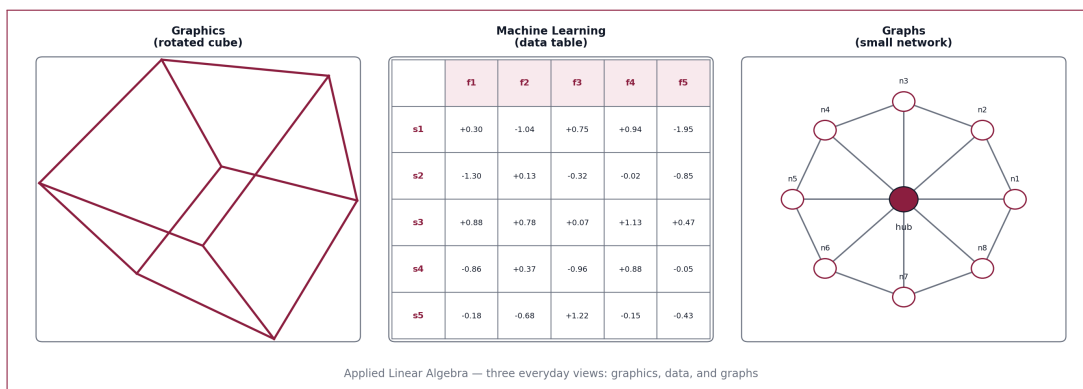


Figure 1: *

Three faces of applied linear algebra: *graphics (rotations), data (matrices), and graphs (networks).*

These are everyday examples where the same linear tools appear in very different forms.

1.1 Why We Need It

When problems get big, we can't think about one variable at a time. We need tools that let us handle thousands or millions of connected quantities. That's exactly what linear algebra does.

It gives us:

- a compact way to store and reason about relationships between many values,

- a language to describe transformations (like rotation or scaling),
- and efficient algorithms that computers can execute very fast.

Another big reason linear algebra is so powerful: many real-world processes, even if not perfectly linear, can be *approximated* as linear systems. This makes them much easier to analyze and simulate.

Note

Linear systems are simple, predictable, and come with *closed-form* solutions. Even if nature is nonlinear, a good linear approximation often works remarkably well, and the trade-off is worth it for the clarity and speed we gain.

Note

By treating curved reality as locally flat, we can project complex phenomena into simpler, linear spaces. The same idea powers computer graphics and data visualization: we map 3D information onto 2D surfaces, where every pixel or data point has a clear, linear position and color.

This shift, from curved and continuous to flat and discrete, is what makes computation possible. It allows us to store, calculate, and render the world efficiently.

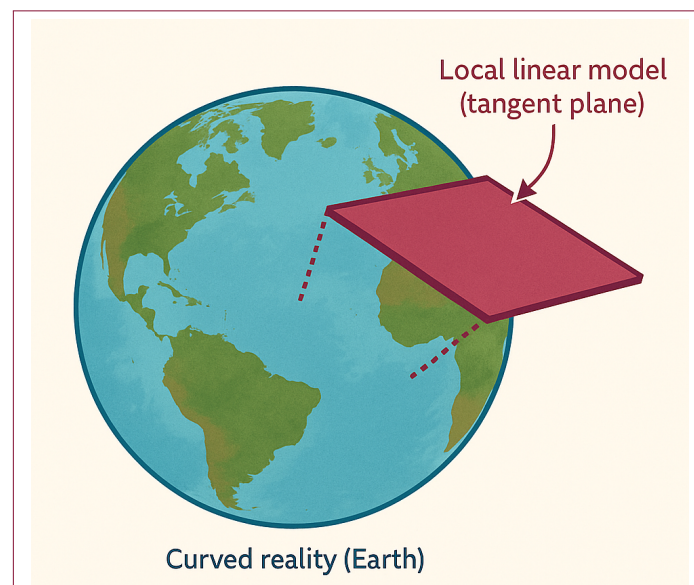


Figure 2: *

From curvature to computation. Even though the Earth is curved, we model it as flat for local calculations. This same logic extends to data: complex structures become manageable once represented in simple, linear, two-dimensional spaces.

Note: Image generated with artificial intelligence for educational purposes.

1.2 From Radar Measurements to Linear Systems

Imagine a highway radar measuring the distance of a moving car at small time intervals. At each time t_i , it records the distance x_i from the radar to the car.

In high-school physics, we know that for uniform acceleration:

$$x(t) = x_0 + vt + \frac{1}{2}at^2$$

where x_0 is the initial position, v the velocity, and a the acceleration.

If we take $n = 5$ distance measurements at different time samples $t_1, t_2, \dots, t_5$, we can write this as a **linear system**:

$$\begin{bmatrix} 1 & t_1 & t_1^2 \\ 1 & t_2 & t_2^2 \\ 1 & t_3 & t_3^2 \\ 1 & t_4 & t_4^2 \\ 1 & t_5 & t_5^2 \end{bmatrix} \begin{bmatrix} x_0 \\ v \\ \frac{1}{2}a \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix}$$

We denote this system compactly as $A\mathbf{x} = \mathbf{b}$, where:

$$A = \begin{bmatrix} 1 & t_1 & t_1^2 \\ \vdots & \vdots & \vdots \\ 1 & t_5 & t_5^2 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_0 \\ v \\ \frac{1}{2}a \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} x_1 \\ \vdots \\ x_5 \end{bmatrix}.$$

Note

In an ideal world, the car's acceleration would be perfectly constant and the radar would give noise-free data. But in reality, acceleration fluctuates slightly, sensors introduce noise, and digital readings have limited precision. Thus, the system $A\mathbf{x} = \mathbf{b}$ may not have an exact solution.

To handle this, we look for an *approximate solution* that minimizes the total squared error:

$$\min_{\mathbf{x}} \|A\mathbf{x} - \mathbf{b}\|^2.$$

This is the principle behind **Least Squares Estimation (LSE)**, the foundation of linear regression and many real-world predictive models.

Note

This simple radar example hides a deep idea: linear algebra allows us to extract structure and predict motion even when our data is imperfect. That's why the same method underlies everything from physics to machine learning and signal processing.

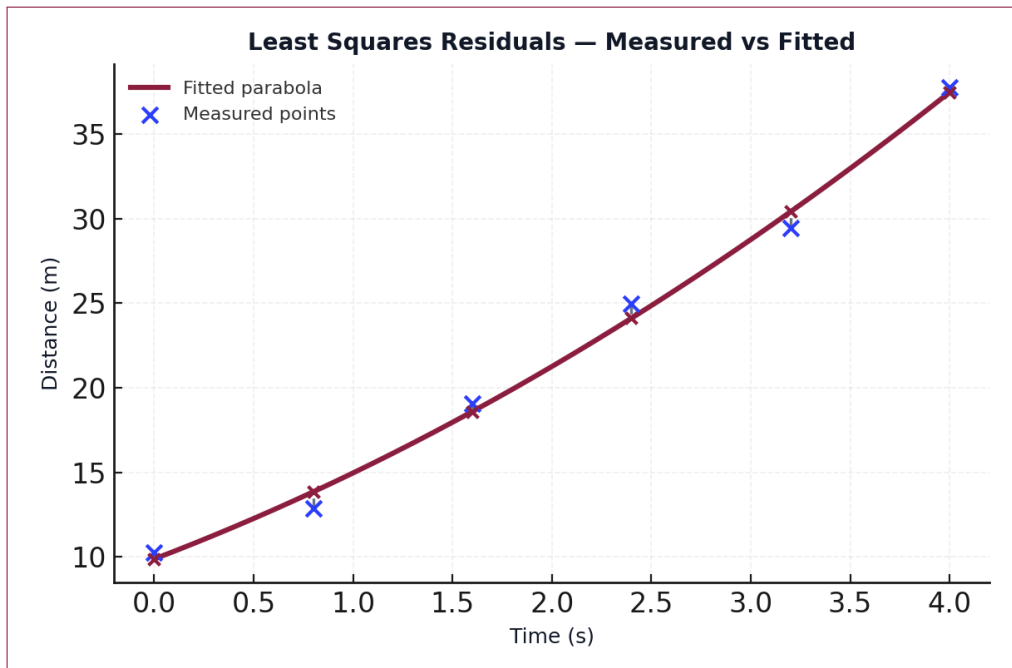


Figure 3: *

Radar distance measurements and quadratic fit. The radar samples the distance of a car over time. The fitted parabola (burgundy) approximates the true motion based on noisy data, using the Least Squares method.

Note

Solving the system gives us the estimated parameters v and a : the velocity and acceleration of the car.

These values tell a meaningful physical story:

- if $a < 0$, the driver was slowing down — perhaps braking right before the radar;
- if $a > 0$, the driver was accelerating — speeding up through the zone;
- if $a \approx 0$, the car kept a steady speed.

From the fitted model, we can also compute the *average velocity* over the interval of measurement, which corresponds to the speed at which the car was observed by the radar.

Thus, a few distance samples and a simple quadratic model allow us to infer both the car's motion and the driver's intent.

2 Linear Algebra Recap

Before diving into least squares, let's recall the key concepts from linear algebra that describe how data, equations, and solutions are structured.

2.1 Vectors, Matrices, and Columns

A vector is an ordered list of numbers, a point in space or a direction. A matrix $A \in \mathbb{R}^{n \times m}$ is a collection of m column vectors, each of length n :

$$A = \begin{bmatrix} | & | & \cdots & | \\ a_1 & a_2 & \cdots & a_m \\ | & | & \cdots & | \end{bmatrix}, \quad a_j \in \mathbb{R}^n.$$

Each column a_j represents one variable's "influence pattern" on all n equations in the system.

Linear combinations and span. A linear combination is any weighted sum:

$$c_1 a_1 + c_2 a_2 + \cdots + c_m a_m.$$

The set of all such combinations is the *column space* (or range) of A :

$$\text{Col}(A) = \{A\mathbf{x} : \mathbf{x} \in \mathbb{R}^m\}.$$

It represents all possible outputs $A\mathbf{x}$ we can obtain by varying the coefficients $\mathbf{x}$.

Linear independence. The columns of A are linearly independent if no column can be written as a combination of the others. Formally,

$$A\mathbf{x} = 0 \quad \Rightarrow \quad \mathbf{x} = 0.$$

If this fails, some variables are redundant, they don't contribute new directions.

Basis and rank. A *basis* is a smallest set of independent vectors that span a space. The number of basis vectors equals the **rank** of the matrix.

$$\text{rank}(A) = \dim(\text{Col}(A)).$$

Example of dependent columns. Let the three column vectors be:

$$a_1 = \begin{bmatrix} 2 \\ 3 \\ 1 \\ 1 \end{bmatrix}, \quad a_2 = \begin{bmatrix} 1 \\ 1 \\ 2 \\ 1 \end{bmatrix}, \quad a_3 = 2a_1 + a_2 = \begin{bmatrix} 5 \\ 7 \\ 4 \\ 3 \end{bmatrix}.$$

Then the matrix

$$A = \begin{bmatrix} 2 & 1 & 5 \\ 3 & 1 & 7 \\ 1 & 2 & 4 \\ 1 & 1 & 3 \end{bmatrix}$$

has columns a_1, a_2, a_3 , but the third one adds no new direction, it is a linear combination of the first two.

Hence:

$$\text{rank}(A) = 2, \quad \text{and} \quad \text{Null}(A) = \{\mathbf{x} : A\mathbf{x} = 0\} \text{ has dimension } 1.$$

Note

Geometrically, a_1 and a_2 span a plane in $\mathbb{R}^3$. The third column a_3 lies inside that same plane, it doesn't expand the span.

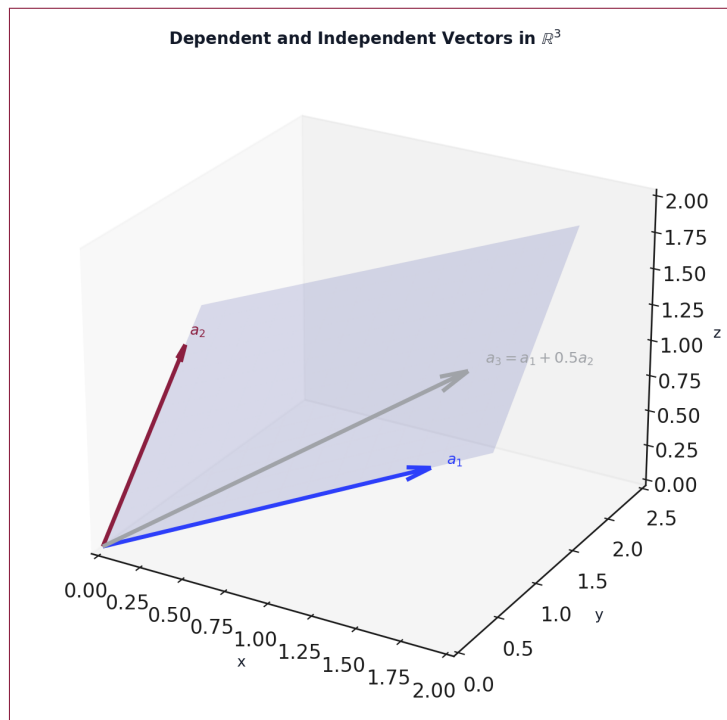


Figure 4: *

Independent vs dependent columns. Vectors a_1 (blue) and a_2 (burgundy) are independent and span a plane in $\mathbb{R}^3$. The third vector $a_3 = a_1 + 0.5a_2$ (gray) lies *in* that plane, so it adds no new direction.

2.2 The Four Fundamental Subspaces

Every matrix $A \in \mathbb{R}^{n \times m}$ defines four geometric subspaces that fully describe how it transforms inputs into outputs. They live in two different worlds: the *input space* $\mathbb{R}^m$ and the *output space* $\mathbb{R}^n$.

$$\begin{aligned}
\textbf{Column space:} \quad \text{Col}(A) &= \{A\mathbf{x} : \mathbf{x} \in \mathbb{R}^m\} \subseteq \mathbb{R}^n \\
\textbf{Null space:} \quad \text{Null}(A) &= \{\mathbf{x} : A\mathbf{x} = 0\} \subseteq \mathbb{R}^m \\
\textbf{Row space:} \quad \text{Row}(A) &= \text{Col}(A^T) \subseteq \mathbb{R}^m \\
\textbf{Left null space:} \quad \text{Null}(A^T) &= \{\mathbf{y} : A^T\mathbf{y} = 0\} \subseteq \mathbb{R}^n
\end{aligned}$$

Dimensional relationships.

- The rank of A is the dimension of both its column and row space:

$$\text{rank}(A) = \dim(\text{Col}(A)) = \dim(\text{Row}(A)).$$

- The nullity (dimension of the null space) complements the rank:

$$\text{rank}(A) + \dim(\text{Null}(A)) = m.$$

- Similarly, for the transpose:

$$\text{rank}(A) + \dim(\text{Null}(A^T)) = n.$$

These equalities express the **Rank–Nullity Theorem**, a balance between how much information is preserved and how much is lost when A acts on a vector.

Note

The column space and null space live in different universes: $\text{Col}(A)$ belongs to the *output space* ($\mathbb{R}^n$), while $\text{Null}(A)$ lives in the *input space* ($\mathbb{R}^m$). Yet they are deeply connected: the dimension of one tells us how much freedom or constraint exists in the other.

Geometric intuition.

- $\text{Col}(A)$ contains all possible outputs $A\mathbf{x}$: what the system can produce.
- $\text{Null}(A)$ contains all inputs that vanish: what the system ignores.
- $\text{Row}(A)$ captures the independent equations among variables.
- $\text{Null}(A^T)$ contains the “invisible” constraints among equations.

2.3 Types of Linear Systems

Consider the linear system $A\mathbf{x} = \mathbf{b}$ with $A \in \mathbb{R}^{n \times m}$, where n is the number of equations and m the number of unknowns.

- Square systems:** $n = m$.

Example:

$$A = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & -1 & 1 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ 0 \\ 4 \end{bmatrix}.$$

Each equation corresponds to a row; each variable to a column.

If $\text{rank}(A) = n$, the system has a unique solution. If not, some equations are redundant or contradictory.

Note

Square systems are the classical case, solvable exactly when the determinant is nonzero.

- **Tall systems:** $n > m$ (more equations than unknowns).

Example:

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 2 & -1 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ 0 \\ 4 \\ 2 \end{bmatrix}.$$

Such systems are usually *overdetermined*. An exact solution may not exist, because the extra equations can be inconsistent. Instead, we seek an approximate solution that minimizes

$$\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|^2,$$

leading to the Least Squares approach.

Note

Tall systems appear naturally in data fitting and regression, where each row corresponds to one observation.

- **Wide systems:** $n < m$ (more unknowns than equations).

Example:

$$A = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 0 & 1 & 3 & -2 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Here there are infinitely many solutions because the system has *fewer equations than variables*. The null space of A describes all possible directions of freedom: if $\mathbf{x}_0$ is one solution, then every $\mathbf{x} = \mathbf{x}_0 + \mathbf{z}$ with $\mathbf{z} \in \text{Null}(A)$ is also a solution.

Note

Wide systems appear in design and optimization problems, where multiple configurations can satisfy the same constraints.

Note

Linear algebra is geometry in disguise: solving $A\mathbf{x} = \mathbf{b}$ means asking “is $\mathbf{b}$ in the column space of A ? If not, which point in $\text{Col}(A)$ is closest to it?”

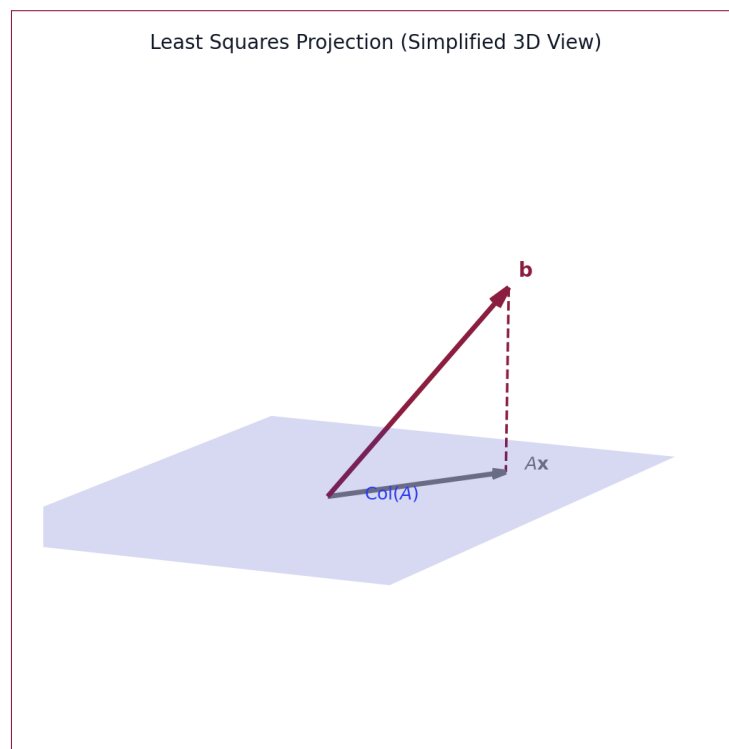


Figure 5: *

Geometric view of least squares. The column space $\text{Col}(A)$ is shown as a plane. The vector $\mathbf{b}$ lies outside it, and the least squares solution finds the closest point $A\mathbf{x}$ on the plane. The dashed line shows the perpendicular residual $\mathbf{r} = \mathbf{b} - A\mathbf{x}$.

3 Square LSE

3.1 Problem Setup

We study linear systems of the form

$$A \mathbf{x} = \mathbf{b}, \quad A \in \mathbb{R}^{n \times n}, \mathbf{x}, \mathbf{b} \in \mathbb{R}^n.$$

When A is square and invertible, the system has a unique exact solution $\mathbf{x} = A^{-1}\mathbf{b}$. In practice, we *do not* form A^{-1} ; we solve $A\mathbf{x} = \mathbf{b}$ via elimination or factorization, which is both faster and numerically safer.

Note

Even for square systems, measurements may contain small errors. If A is near-singular or data are noisy, least-squares ideas remain relevant (e.g., regularization, conditioning). But the default for square, well-posed systems is direct solve (elimination/factorization). We will discuss more on least-square idea soon.

3.2 Augmented Form and Row Operations

To systematically solve a square system

$$A\mathbf{x} = \mathbf{b}, \quad A \in \mathbb{R}^{n \times n},$$

we often work with its **augmented matrix**:

$$[A | \mathbf{b}] = \left[\begin{array}{cccc|c} a_{11} & a_{12} & \dots & a_{1n} & b_1 \\ a_{21} & a_{22} & \dots & a_{2n} & b_2 \\ \vdots & \vdots & & \vdots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} & b_n \end{array} \right].$$

Each row encodes one equation; the last column is the right-hand side vector $\mathbf{b}$.

Why augmentation? Writing $[A | b]$ lets us perform the same algebraic operations simultaneously on the coefficients and on the right-hand side, ensuring consistency throughout elimination.

Elementary Row Operations

There are three fundamental row operations:

- (1) Row exchange: $R_i \leftrightarrow R_j$
- (2) Row scaling: $R_i \leftarrow c R_i, \quad c \neq 0$
- (3) Row replacement: $R_i \leftarrow R_i + c R_j$

Each of these corresponds to left-multiplying the system by an invertible matrix. Thus, all

elementary row operations preserve the solution set of the system $A\mathbf{x} = \mathbf{b}$.

Note

Applying row operations to $[A|b]$ is equivalent to forming $E_k \cdots E_2 E_1 A = U$ and $E_k \cdots E_2 E_1 b = c$, where each E_i is an elementary (invertible) matrix.

Structure of the elementary matrices. Each elementary row operation corresponds to a specific type of matrix E_i that acts on the left of A (and on b) to produce a new equivalent system. There are exactly three families of such matrices:

(1) Row-exchange matrices

A *row-exchange matrix* swaps two rows of a matrix. It is obtained from the identity matrix I by interchanging the same two rows. For instance, to swap R_1 and R_3 in a 3×3 system:

$$E_{\text{swap}} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}.$$

Multiplying by this E_{swap} simply permutes the rows of A :

$$E_{\text{swap}}A = \begin{bmatrix} \text{row}_3(A) \\ \text{row}_2(A) \\ \text{row}_1(A) \end{bmatrix}.$$

This matrix is invertible, since applying the same swap again returns the original order: $E_{\text{swap}}^{-1} = E_{\text{swap}}$. Its determinant is -1 , because the operation corresponds to a single transposition.

(2) Row-scaling matrices

A *scaling matrix* multiplies one row by a nonzero scalar c . It is the identity matrix with the pivot entry on the diagonal replaced by c :

$$E_{\text{scale}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

When we compute $E_{\text{scale}}A$, only the second row of A is scaled by c . This matrix is invertible whenever $c \neq 0$, with inverse given by

$$E_{\text{scale}}^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \frac{1}{c} & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The determinant of E_{scale} equals c , reflecting the scaling of one row by that constant.

(3) Row-replacement matrices

A *replacement matrix* adds a multiple of one row to another: $R_i \leftarrow R_i + cR_j$. This is represented by the identity matrix with one extra entry c in position (i, j) (row i , column j):

$$E_{\text{replace}} = \begin{bmatrix} 1 & 0 & 0 \\ c & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Applying this to A performs the replacement operation:

$$E_{\text{replace}}A = \begin{bmatrix} \text{row}_1(A) \\ \text{row}_2(A) + c\text{row}_1(A) \\ \text{row}_3(A) \end{bmatrix}.$$

This matrix is invertible because the inverse operation is $R_i \leftarrow R_i - cR_j$, which corresponds to

$$E_{\text{replace}}^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ -c & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The determinant of E_{replace} is 1, since the main diagonal consists only of ones.

Numerical Example. Consider the system:

$$\begin{cases} x_1 + 2x_2 + 3x_3 = 9, \\ 2x_1 + 3x_2 + 7x_3 = 24, \\ x_1 - x_2 + 2x_3 = 5. \end{cases}$$

We write it in augmented form:

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 9 \\ 2 & 3 & 7 & 24 \\ 1 & -1 & 2 & 5 \end{array} \right].$$

Step 1: Eliminate the first column below the diagonal. Subtract $2R_1$ from R_2 , and subtract R_1 from R_3 :

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 9 \\ 0 & -1 & 1 & 6 \\ 0 & -3 & -1 & -4 \end{array} \right].$$

Step 2: Eliminate the entry below the pivot in column 2. Replace $R_3 \leftarrow R_3 - 3R_2$:

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 9 \\ 0 & -1 & 1 & 6 \\ 0 & 0 & -4 & -22 \end{array} \right].$$

We have now achieved *upper-triangular form*.

Step 3: Back substitution.

$$\begin{cases} -4x_3 = -22 \Rightarrow x_3 = 5.5, \\ -x_2 + x_3 = 6 \Rightarrow x_2 = -0.5, \\ x_1 + 2x_2 + 3x_3 = 9 \Rightarrow x_1 = -3.5. \end{cases}$$

Thus,

$$\mathbf{x} = \begin{bmatrix} -3.5 \\ -0.5 \\ 5.5 \end{bmatrix}.$$

Note

Every row operation changes the algebraic representation of the system but not its solution. The augmented matrix format keeps all steps synchronized between A and b .

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 9 \\ 2 & 3 & 7 & 24 \\ 1 & -1 & 2 & 5 \end{array} \right] \rightarrow R_2 \leftarrow R_2 - 2R_1, R_3 \leftarrow R_3 - R_1$$

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 9 \\ 0 & -1 & 1 & 6 \\ 0 & -3 & -1 & -4 \end{array} \right] \xrightarrow{R_3 \leftarrow R_3 - 3R_2} \left[\begin{array}{ccc|c} 1 & 2 & 3 & 9 \\ 0 & -1 & 1 & 6 \\ 0 & 0 & -4 & -22 \end{array} \right]$$

Note

From a computational perspective, row operations correspond to left-multiplying by simple matrices (permutations, scalings, eliminations). This view generalizes naturally into LU factorization, which is essentially a structured product of such elementary operations.

3.3 Gaussian Elimination (GE)

We now formalize the procedure introduced earlier for transforming the augmented matrix $[A | b]$ into an equivalent upper-triangular system.

As discussed in the previous section, Gaussian Elimination applies a sequence of elementary row operations to zero out entries below the pivots, producing:

$$[A | b] \xrightarrow{\text{elimination}} [U | c], \quad U \text{ upper-triangular.}$$

Each step can be viewed as multiplying the system on the left by an *elementary matrix* E_i , so

that the full elimination sequence satisfies

$$E_k E_{k-1} \cdots E_1 A = U, \quad E_k E_{k-1} \cdots E_1 b = c.$$

Because each E_i is invertible, the transformed system has the *same solution set* as the original.

Note

In practice, we rarely form the E_i explicitly. Instead, we apply the operations directly to A and b , which is what numerical solvers (e.g. `numpy.linalg.solve`) do internally.

Algorithmic structure and computational cost. Gaussian Elimination proceeds in two distinct phases:

1. **Forward elimination:** In this phase we eliminate the entries below each pivot, column by column. For every pivot row, we need to update all rows beneath it, roughly $\mathcal{O}(n^2)$ pairs of rows across the entire process. For each such pair, we traverse the remaining entries of the row (about $\mathcal{O}(n)$ work) to perform the subtraction and scaling. Together, this gives a total computational cost proportional to

$$\mathcal{O}(n^2) \times \mathcal{O}(n) = \mathcal{O}(n^3).$$

This is why forward elimination dominates the total runtime.

2. **Back substitution:** Once A has been reduced to upper-triangular form U , we solve $U\mathbf{x} = c$. Starting from the last equation, each variable depends on all those already computed, giving a total cost of order $\mathcal{O}(n^2)$, one pass through a triangular matrix.

Total cost: $\mathcal{O}(n^3)$ (forward elimination) + $\mathcal{O}(n^2)$ (back substitution) $\approx \mathcal{O}(n^3)$.

Note

Forward elimination is the bottleneck of Gaussian Elimination. Its cubic growth means that doubling the matrix dimension multiplies runtime roughly by eight. This motivates the search for faster structured algorithms and iterative solvers when n becomes very large.

3.4 Permutation and Factorization: from Row Operations to LU Decomposition

In Gaussian Elimination, every step corresponds to multiplying A by a sequence of elementary matrices E_i . When we examine the structure of these operations more closely, a clear pattern emerges:

- **Row-exchange operations** correspond to multiplying by a *permutation matrix* P .

- **Row-scaling** and **row-replacement** operations, when accumulated in reverse order, form a *lower-triangular matrix* L .

Together, these observations lead to the famous **LU (or PLU) decomposition** of a square matrix.

Permutation matrices and row exchanges

A permutation matrix P is obtained by permuting the rows of the identity matrix. Multiplying a matrix A on the left by P simply reorders its rows.

For example, swapping R_1 and R_3 in a 3×3 system:

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \quad PA = \begin{bmatrix} \text{row}_3(A) \\ \text{row}_2(A) \\ \text{row}_1(A) \end{bmatrix}.$$

Permutation matrices are always invertible since $P^{-1} = P^T$, and their determinant is either $+1$ or -1 , depending on whether the permutation involves an even or odd number of swaps.

Note

Partial pivoting in Gaussian Elimination corresponds exactly to left-multiplying by such permutation matrices. At each step, we choose the pivot row with the largest magnitude entry and swap it into position to improve numerical stability.

To see why row exchanges (and therefore permutation matrices) are essential, consider a system where the pivot element is zero:

$$A = \begin{bmatrix} 0 & 2 & 3 \\ 1 & 4 & 5 \\ 2 & 8 & 10 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 6 \\ 7 \\ 8 \end{bmatrix}.$$

If we try to start Gaussian Elimination from the first column, the pivot $a_{11} = 0$ makes it impossible to eliminate the entries below it: division by zero would occur immediately.

No elimination step can proceed, so the algorithm stalls at the very first step.

With row exchange: We swap the first and second rows to bring a nonzero pivot into position:

$$P = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad PA = \begin{bmatrix} 1 & 4 & 5 \\ 0 & 2 & 3 \\ 2 & 8 & 10 \end{bmatrix}.$$

Now elimination can proceed normally.

After the swap, we can eliminate entries below the pivot:

$$R_3 \leftarrow R_3 - 2R_1 \quad \Rightarrow \quad \begin{bmatrix} 1 & 4 & 5 \\ 0 & 2 & 3 \\ 0 & 0 & 0 \end{bmatrix}.$$

Lower matrices from elimination steps

Now recall that every elimination step E_i used to zero out entries below a pivot is itself a lower-triangular matrix with ones on the diagonal:

$$E_i = \begin{bmatrix} 1 & 0 & 0 \\ -\ell_{21} & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \text{where } \ell_{21} \text{ is the multiplier used to eliminate below the pivot.}$$

When we perform a sequence of such eliminations, we actually multiply A on the left by

$$E_k E_{k-1} \cdots E_1 A = U.$$

If we define

$$L = (E_k E_{k-1} \cdots E_1)^{-1},$$

then L is also lower-triangular, with the elimination coefficients ℓ_{ij} stored below the diagonal. Thus, we can rewrite the entire elimination process as

$$A = LU \quad \text{or, with pivoting,} \quad PA = LU.$$

Why L is lower-triangular

Each elimination step only modifies rows below the current pivot, which means all the corresponding E_i 's are lower-triangular with 1s on the diagonal. The inverse of a product of such matrices remains lower-triangular, so their cumulative effect forms the matrix L . Its structure encodes exactly the multipliers used during elimination.

For a 3×3 system:

$$L = \begin{bmatrix} 1 & 0 & 0 \\ \ell_{21} & 1 & 0 \\ \ell_{31} & \ell_{32} & 1 \end{bmatrix}, \quad U = \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}.$$

Note

Once L , U , and P are computed, each new right-hand side $\mathbf{b}$ can be solved efficiently by two triangular solves:

$$Ly = P\mathbf{b}, \quad U\mathbf{x} = y.$$

This reuse of L and U makes LU decomposition one of the most fundamental building blocks in numerical linear algebra.

Computational perspective.

- The **factorization step**, computing L and U from A , requires about $\mathcal{O}(n^3)$ operations, exactly like one full Gaussian elimination. This cost is paid only once for a given matrix A .
- Each subsequent **solve** for a new right-hand side $\mathbf{b}$ only requires two triangular systems:

$$Ly = P\mathbf{b} \quad \text{and} \quad U\mathbf{x} = y.$$

Because L and U are triangular, these steps involve only forward and backward substitutions, costing about $\mathcal{O}(n^2)$ operations per right-hand side.

Overall complexity. If we need to solve the same system $A\mathbf{x} = \mathbf{b}$ for multiple right-hand sides $\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_\ell$, the total cost becomes

$$\boxed{\mathcal{O}(n^3) + \ell \mathcal{O}(n^2)}.$$

This is a dramatic improvement compared to performing a separate Gaussian elimination for each $\mathbf{b}$, which would cost $\ell \mathcal{O}(n^3)$ in total.

When $\ell = \mathcal{O}(n)$, the total cost remains $\mathcal{O}(n^3)$ instead of growing to $\mathcal{O}(n^4)$, a difference of several orders of magnitude for large systems. This efficiency explains why LU factorization is preferred in practice whenever A stays fixed but the right-hand side changes.

3.5 Block Gaussian Elimination

In many real-world systems, the matrix A has a natural block structure, for example, when variables can be grouped into two or more interacting subsystems. In such cases, it is often more efficient and conceptually cleaner to perform **Gaussian Elimination at the block level**.

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \end{bmatrix}.$$

We can think of this as two coupled systems:

$$\begin{cases} A_{11}\mathbf{x}_1 + A_{12}\mathbf{x}_2 = \mathbf{b}_1, \\ A_{21}\mathbf{x}_1 + A_{22}\mathbf{x}_2 = \mathbf{b}_2. \end{cases}$$

Step 1: Eliminate one block of variables

We start by solving the first equation for $\mathbf{x}_1$ (assuming A_{11} is invertible):

$$\mathbf{x}_1 = A_{11}^{-1}(\mathbf{b}_1 - A_{12}\mathbf{x}_2).$$

Substitute this expression into the second equation to eliminate $\mathbf{x}_1$:

$$A_{21}A_{11}^{-1}(\mathbf{b}_1 - A_{12}\mathbf{x}_2) + A_{22}\mathbf{x}_2 = \mathbf{b}_2.$$

Simplifying gives:

$$(A_{22} - A_{21}A_{11}^{-1}A_{12})\mathbf{x}_2 = \mathbf{b}_2 - A_{21}A_{11}^{-1}\mathbf{b}_1.$$

Step 2: The Schur complement

The matrix

$$S = A_{22} - A_{21}A_{11}^{-1}A_{12}$$

is called the **Schur complement** of A_{11} in A . It represents the “effective system” seen by the remaining variables $\mathbf{x}_2$ after $\mathbf{x}_1$ has been eliminated.

Once $\mathbf{x}_2$ is solved from this smaller system, we can recover $\mathbf{x}_1$ using

$$\mathbf{x}_1 = A_{11}^{-1}(\mathbf{b}_1 - A_{12}\mathbf{x}_2).$$

Interpretation

This block procedure mirrors scalar Gaussian Elimination, but works entirely with submatrices instead of single elements. In effect:

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \longrightarrow \begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{22} - A_{21}A_{11}^{-1}A_{12} \end{bmatrix}.$$

The lower-left block A_{21} is “zeroed out”, and the bottom-right block becomes the Schur complement.

This approach generalizes to multiple blocks, and is widely used in:

- **Sparse linear systems** — where submatrices correspond to local couplings;
- **Multiphysics problems** — where A_{11} and A_{22} represent separate physical models;
- **Parallel computing** — because block operations can be distributed among processors.

Computational remark. Block elimination is especially useful when A_{11} (or A_{22}) has a structure that allows fast inversion or factorization. For example, if A_{11} is diagonal or triangular, then forming the Schur complement is cheap. In practice, we never compute A_{11}^{-1} explicitly; instead, we apply it through forward or backward solves, just like in LU decomposition.

3.6 Computing the Matrix Inverse in Practice

In numerical computation, the inverse of a matrix A is almost never formed by applying a direct algebraic formula. Instead, it is computed using the same machinery as Gaussian Elimination and LU factorization.

$$AX = I_n, \quad \text{where } X = A^{-1}.$$

This means that the i -th column of A^{-1} is the solution to the linear system

$$A\mathbf{x}_i = \mathbf{e}_i,$$

where $\mathbf{e}_i$ is the i -th column of the identity matrix I_n .

Practical procedure

In practice, computing A^{-1} proceeds as follows:

1. Perform an LU factorization (with pivoting if needed):

$$PA = LU.$$

This step costs $\mathcal{O}(n^3)$ operations.

2. Solve the n triangular systems:

$$LY = PI_n, \quad UX = Y.$$

Each column of I_n acts as a separate right-hand side. Since each triangular solve costs $\mathcal{O}(n^2)$, solving for all n columns requires another $\mathcal{O}(n^3)$ operations in total.

Hence, forming the full inverse explicitly costs

$$\boxed{\mathcal{O}(n^3) \text{ (factorization)} + \mathcal{O}(n^3) \text{ (n solves)} = \mathcal{O}(n^3)}.$$

Note

If we denote by ℓ the number of right-hand sides, the general cost formula is

$$\mathcal{O}(n^3) + \ell \mathcal{O}(n^2).$$

For computing A^{-1} , we have $\ell = n$, giving again $\mathcal{O}(n^3)$. This is vastly more efficient than recomputing a new factorization for each column (which would cost $\mathcal{O}(n^4)$).

3.7 Python Snippets (Direct Solve)

One right-hand side.

```
import numpy as np

A = np.array([[2., 1., 1.],
              [4., -6., 0.],
              [-2., 7., 2.]])
b = np.array([5., -2., 9.])

# Robust direct solve (uses LU with pivoting under the hood)
x = np.linalg.solve(A, b)
print(x)
```

Multiple right-hand sides.

```
# If B has multiple columns (each a right-hand side), solve all at once.
B = np.column_stack([b, 2*b + 1])
X = np.linalg.solve(A, B) # columns of X solve A X = B
```

Avoid explicit inverse.

```
# Avoid x = np.linalg.inv(A) @ b # slower + less stable
# Prefer np.linalg.solve(A, b)
```

4 Exploiting Structure in Solving Linear Systems

Real-world matrices often contain patterns that can be exploited to make Gaussian Elimination dramatically faster. Instead of performing $\mathcal{O}(n^3)$ operations blindly, we can tailor the algorithm to the matrix structure and reduce complexity to nearly linear time.

4.1 Example 1: Arrow Matrices

An **arrow matrix** has a simple but powerful structure: a diagonal block plus one additional row and column coupling all entries.

$$A = \begin{bmatrix} D & \mathbf{a}_1 \\ \mathbf{a}_2^\top & \alpha \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} \mathbf{x}_1 \\ \chi \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} \mathbf{b}_1 \\ \beta \end{bmatrix}.$$

Here:

$$D = \text{diag}(d_1, d_2, \dots, d_n), \quad d_i \neq 0, \quad \alpha \neq 0,$$

and $\mathbf{a}_1, \mathbf{a}_2$ are column vectors (having n components each).

Elimination idea. Because D is diagonal, the first n equations are trivially independent:

$$d_i x_i + a_{1,i} \chi = b_i, \quad i = 1, \dots, n.$$

Each of them can be rearranged as

$$x_i = \frac{b_i - a_{1,i} \chi}{d_i}.$$

Substituting these into the last equation (the bottom row of $A\mathbf{x} = \mathbf{b}$):

$$\sum_{i=1}^n a_{2,i} x_i + \alpha \chi = \beta,$$

we obtain

$$\alpha \chi - \sum_{i=1}^n \frac{a_{1,i} a_{2,i}}{d_i} \chi = \beta - \sum_{i=1}^n \frac{a_{2,i} b_i}{d_i}.$$

Define the reduced coefficients:

$$\alpha' = \alpha - \sum_{i=1}^n \frac{a_{1,i} a_{2,i}}{d_i}, \quad \beta' = \beta - \sum_{i=1}^n \frac{a_{2,i} b_i}{d_i}.$$

Then

$$\boxed{\chi = \frac{\beta'}{\alpha'}}.$$

Once χ is known, all other x_i follow directly from

$$x_i = \frac{b_i - a_{1,i} \chi}{d_i}.$$

Computational cost. The above procedure only requires:

- n scalar multiplications and divisions to compute α' and β' ;
- another n updates to recover all x_i .

Hence the total cost is linear:

$$\boxed{\mathcal{O}(n)}.$$

This is a massive improvement compared to the $\mathcal{O}(n^3)$ of generic Gaussian Elimination.

Note

The key is to recognize that the diagonal block D can be “inverted” elementwise, avoiding any full elimination. The extra row and column contribute only low-rank updates.

4.2 Example 2: Low-Rank Modifications and the Sherman–Morrison–Woodbury Formula

In many real applications, we need to repeatedly solve linear systems where the coefficient matrix changes slightly between iterations (remember the discussion in the previous week about the quasi-newton method):

$$\tilde{A}\mathbf{x} = \mathbf{b}, \quad \tilde{A} = A + UV^T,$$

where $U, V \in \mathbb{R}^{n \times k}$ and $k \ll n$. The matrix $\tilde{A}$ is called a *low-rank modification* of A .

If we already know the LU (or inverse) of A , it is wasteful to recompute everything from scratch. Instead, we can exploit the low-rank structure to update the solution efficiently.

Motivating idea. Suppose we have solved

$$A\mathbf{x}_0 = \mathbf{b},$$

and now want the solution for $\tilde{A}\mathbf{x} = \mathbf{b}$. We can think of $\tilde{A}$ as A plus a correction of rank k . If k is small, we should only pay $\mathcal{O}(nk^2)$ cost, not $\mathcal{O}(n^3)$ again.

From block elimination to the Schur complement. Recall that for a block matrix

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad \text{the Schur complement is } S = A_{22} - A_{21}A_{11}^{-1}A_{12}.$$

This reduction eliminates one block and leads naturally to the idea of updating an existing factorization without refactoring everything.

The same logic applies here, with U and V acting as low-rank “corrections.”

Derivation of the Woodbury identity. To understand where the formula comes from, we start with the system

$$(A + UV^T)\mathbf{x} = \mathbf{b}.$$

We want to express $\mathbf{x}$ in terms of quantities involving A^{-1} , without explicitly inverting the entire matrix $A + UV^T$.

Step 1 — Introducing auxiliary variables.

The term $UV^T\mathbf{x}$ couples the two matrices. We define an auxiliary variable

$$\mathbf{y} = V^T\mathbf{x}.$$

Substituting this into the equation gives:

$$A\mathbf{x} + U\mathbf{y} = \mathbf{b}.$$

We can also write the definition of $\mathbf{y}$ as another equation:

$$V^T\mathbf{x} - \mathbf{y} = \mathbf{0}.$$

Together, these two equations form a larger block system:

$$\begin{bmatrix} A & U \\ V^T & -I_k \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \end{bmatrix} = \begin{bmatrix} \mathbf{b} \\ \mathbf{0} \end{bmatrix}.$$

This block matrix compactly encodes both relations: the first row corresponds to $A\mathbf{x} + U\mathbf{y} = \mathbf{b}$, and the second to $V^T\mathbf{x} = \mathbf{y}$.

Step 2 — Eliminating $\mathbf{x}$ or $\mathbf{y}$.

From the first block row, we have

$$A\mathbf{x} = \mathbf{b} - U\mathbf{y} \quad \Rightarrow \quad \mathbf{x} = A^{-1}(\mathbf{b} - U\mathbf{y}).$$

Substitute this into the second equation:

$$V^T A^{-1}(\mathbf{b} - U\mathbf{y}) = \mathbf{y}.$$

Rearranging terms:

$$(I_k + V^T A^{-1}U)\mathbf{y} = V^T A^{-1}\mathbf{b}.$$

Thus, we can solve for $\mathbf{y}$ as

$$\boxed{\mathbf{y} = (I_k + V^T A^{-1}U)^{-1}(V^T A^{-1}\mathbf{b}).}$$

Step 3 — Substituting back to get $\mathbf{x}$.

Plugging this $\mathbf{y}$ into $\mathbf{x} = A^{-1}(\mathbf{b} - U\mathbf{y})$ gives:

$$\mathbf{x} = A^{-1}\mathbf{b} - A^{-1}U(I_k + V^T A^{-1}U)^{-1}V^T A^{-1}\mathbf{b}.$$

Since $\mathbf{x}$ satisfies $(A + UV^\top)\mathbf{x} = \mathbf{b}$, this relation must hold for all $\mathbf{b}$, and therefore:

$$(A + UV^\top)^{-1} = A^{-1} - A^{-1}U(I_k + V^\top A^{-1}U)^{-1}V^\top A^{-1}.$$

Computational benefit.

- Inverting A (or computing its LU) costs $\mathcal{O}(n^3)$ once.
- Each low-rank update only costs $\mathcal{O}(nk^2)$ operations, since $(I_k + V^\top A^{-1}U)$ is a small $k \times k$ system.

When k is small (say 1 or 2), this is an enormous saving over recomputing the entire factorization.

Note

For $k = 1$, the formula reduces to the **Sherman–Morrison identity**:

$$(A + \mathbf{u}\mathbf{v}^\top)^{-1} = A^{-1} - \frac{A^{-1}\mathbf{u}\mathbf{v}^\top A^{-1}}{1 + \mathbf{v}^\top A^{-1}\mathbf{u}}.$$

This is often used in adaptive systems.